Toolchain, board, broker, and project kickoff

TinyGo, a blinking LED, a local MQTT broker, team setup

Dinu-Ștefan Rusu MEC

Lab objectives

Put code on a device

Install TinyGo, build the blink program for your board or for the simulator, flash it, and read what it prints on the serial console.

Run your own broker

Install Mosquitto, publish and subscribe from the command line, then agree on a project idea that meets all five requirements.

Time
2 hours

Present
To the lab instructor, at the end of this lab

Bring a board if you have one. If you do not, Task III replaces Task II and is worth the same.

Project folder

1. Work in your own copy of the Flutter app from ADMD. Every lab adds to it.
2. Create the folder this lab writes into:

```
mkdir -p device/blink
```

3. Present your work to the lab instructor at the end of the lab.

Hint

Labs are individual. The team you form in Task V is for the project.

Install Go and TinyGo

```
# macOS
brew install go
brew tap tinygo-org/tools && brew install tinygo

# Windows, with Scoop
scoop install go tinygo

# Debian and Ubuntu: Go from go.dev/dl, TinyGo from its releases page
sudo dpkg -i tinygo_*.deb
tinygo version
```

Install Go first: TinyGo reuses the Go standard library and the module cache. If `tinygo version` is not found, the TinyGo `bin` folder is not on your PATH.

Board and TinyGo target

1. Pick one: a Raspberry Pi Pico, an ESP32-C3 board, or an ESP32-S3 board. With no hardware, go to Task III.
2. List what your TinyGo can build for:

```
tinygo targets
```

3. Take the target name from your board's page on the TinyGo site. The Pico is `pico`, a plain ESP32-C3 board is `esp32c3`.

Done when

- [ ] `tinygo version` prints a version.
- [ ] Your target appears in the output of `tinygo targets`.
- [ ] You can name the target for your board.

Watch out

Board variants have their own targets, such as `xiao-esp32c3`. Check the board page.

Blink program: full listing

```
package main
import "machine"
import "time"
func main() {
    led := machine.LED
    led.Configure(machine.PinConfig{Mode: machine.PinOutput})
    for {
        led.Set(!led.Get())
        println("blink")
        time.Sleep(500 * time.Millisecond)
    }
}
```

Save it as `device/blink/main.go`. Use the LED pin your board page names.

Flash the board and read the console

1. Plug the board in with a data cable, not a charge-only cable.
2. Build and flash in one command:

```
tinygo flash -target=pico ./device/blink
```

3. Watch the LED, then read what the program prints:

```
tinygo monitor
```

4. Stop the monitor with Ctrl-C.

Done when

- [ ] The LED blinks about once a second.
- [ ] `tinygo monitor` prints one line per blink.
- [ ] The board blinks when you present.

Wokwi simulation without a board

1. On wokwi.com open the ESP32 custom application template, then set the board to ESP32-C3 DevKitM-1.
2. Add an LED and a 220 ohm resistor from the parts menu, wired to a free GPIO pin. Use that pin in the code.
3. Build the firmware on your laptop:

```
tinygo build -o blink.elf \
  -target=esp32c3 ./device/blink
```

4. Press F1, choose *Upload Firmware and Start Simulation*, and pick `blink.elf`.

Done when

- ☐ The simulated LED blinks.
- ☐ The Wokwi serial monitor prints one line per blink.
- ☐ The simulation runs when you present.

Install the Mosquitto broker

```
# macOS
brew install mosquitto

# Debian and Ubuntu
sudo apt install mosquitto mosquitto-clients

mosquitto -v          # foreground, port 1883, one line per event
```

Hint

Mosquitto 2 and later accept local connections only until you configure a listener. In this lab that default is what you want.

Publish and subscribe from two terminals

1. Leave the broker running. In a second terminal, subscribe to a wildcard topic:

```
mosquitto_sub -h localhost -t mec/lab1/#
```

2. In a third terminal, publish to a subtopic:

```
mosquitto_pub -h localhost \
  -t mec/lab1/hello -m "1"
```

3. Publish three more messages, changing the subtopic each time.

Done when

- ☐ The subscriber prints every message the publisher sends.
- ☐ Messages from different subtopics all reach the wildcard subscription.
- ☐ The broker log lists both clients connecting.

Team and project idea

1. Form a project team of at most three. Fill one row in the team sheet in the Teams channel: the names, and the idea in one paragraph.
2. Add one line for each of the five requirements on the next slide.
3. Show it to me during this lab. Milestone 1 is the team and the idea approved.

Done when

- ☐ The sheet row is filled.
- ☐ The row names the sensor, the transport, and the conflict rule.
- ☐ The idea is approved.

Project requirements

Requirement	True at the defense
Device component	A TinyGo program on a board or in Wokwi, reading a sensor.
Device to phone	The reading arrives over MQTT or BLE and the app shows it live.
Offline-first	The app works with no network, syncs later, states its conflict rule.
One boundary	Either gRPC with a Go server, or FFI, or a platform channel.
A measured claim	One number your team measured, with the method.

All five, or the project is not approved.

Flashing and serial port errors

`permission denied: /dev/ttyUSB0`	Your user is not in the serial port group. Add yourself to `dialout` on Debian and Ubuntu, or `uucp` on Arch, then log out and back in. Do not flash with sudo.
`no serial ports found`	The cable carries power only, or the board is not in its bootloader. Swap the cable, then hold the BOOT button while plugging the board in and release it.
`unknown target or board name`	The string after `-target=` is not one TinyGo knows. Copy it from `tinygo targets` or from the board's page.

Broker connection errors

`Error: Connection refused`	Nothing is listening on port 1883. Start the broker in a terminal with `mosquitto -v` and leave it running while you test.
`Connection refused from another machine`	The default configuration binds to localhost. Add `listener 1883` and `allow_anonymous true` to `mosquitto.conf`, and open the port in the local firewall. Lab network only.
`Address already in use`	A broker is already running, often as a system service. Use that one, or stop the service before starting your own.

What you present

Proof it ran

The board blinking, or the Wokwi simulation running, with the serial output next to it.

Proof it talks

The two terminals from Task IV, subscriber and publisher, with the messages visible.

Also

The board and the target name you used, and your project team’s row in the sheet.

Presentation

Blink and serial output shown live · idea approved